

Python per Statistica

Cheat Sheet da esami ed esercizi reali

Cheat Sheet Python per Statistica (Esami/Esercizi)

> Basata su codice usato in esercizi reali con pandas, numpy, scipy, matplotlib e statsmodels

1. Caricare e ispezionare dataset

```
import pandas as pd

dati = pd.read_csv("nome_file.csv")    # carica CSV
dati.head()                           # prime 5 righe
dati.columns                           # nomi colonne
dati.dtypes                            # tipo di ogni colonna
dati.shape                             # (righe, colonne)
```

2. Valori mancanti

```
dati.isnull().sum()                    # mancanti per ogni colonna
dati.isnull().sum().sum()               # totale mancanti

# con illoc (se non ricordi i nomi colonna):
for i in range(len(dati.columns)):
    print(dati.columns[i], ":", dati.iloc[:, i].isna().sum())
```

3. Frequenze e tabelle

```
# Frequenze assolute
dati["colonna"].value_counts()

# Frequenze relative (normalizzate)
dati["colonna"].value_counts(normalize=True)

# Frequenze relative cumulate
dati["colonna"].value_counts(normalize=True).sort_index().cumsum()

# Frequenze relative congiunte (tabelle a doppia entrata)
pd.crosstab(dati["col1"], dati["col2"], normalize=True, margins=True)
# margins=True aggiunge riga/colonna "All" con i totali marginali

# Trovare il primo n tale che la freq cumulativa supera 0.95:
freq_cum = dati["colonna"].value_counts(normalize=True).sort_index().cumsum()
n = freq_cum[freq_cum >= 0.95].index[0]
```

****Esempio reale da auto.csv:****

```
auto.numero_occupanti.value_counts(normalize=True).sort_index()
```

4. Selezionare righe/colonne

```
# Filtrare righe (condizione)
dati[dati["colonna"] == valore]
dati[dati["colonna"] > soglia]
dati[dati["colonna"].isna()]          # righe con NaN

# Filtrare e selezionare solo una colonna
dati[dati["allarme"] == 1]["richieste"]

# Con .loc (righe, colonne)
dati.loc[dati["allarme"] == 1, "richieste"]

# Filtrare per più valori
```

```
dati[dati["mese"].isin([12, 1, 2])]    # mesi invernali

# .iloc per posizione
dati.iloc[:, 0]    # prima colonna (tutte le righe)
dati.iloc[0, :]    # prima riga (tutte le colonne)
dati.iloc[1, 2]    # riga 1, colonna 2
```

5. Media, varianza, correlazione, indici di eterogeneità

```
import numpy as np

dati["colonna"].mean()            # media
dati["colonna"].var()             # varianza campionaria
dati["colonna"].std()             # deviazione standard
dati["colonna"].quantile(0.95)    # 95° percentile

np.mean(array)                   # media su array numpy
np.var(array, ddof=1)             # varianza campionaria (ddof=1)
np.var(array, ddof=0)             # varianza popolazionistica
np.median(array)                 # mediana
np.corrcoef(x, y)[0,1]           # correlazione tra array

# Correlazione su DataFrame
dati["col1"].corr(dati["col2"])

# Media pesata
np.average(valori, weights=freq)

# *** INDICE DI GINI (normalizzato) ***
# Misura di eterogeneità: 0=tutti uguali, 1=tutti diversi
freq_rel = dati["colonna"].value_counts(normalize=True)
k = len(freq_rel)
gini = 1 - np.sum(freq_rel**2)
gini_norm = gini / (1 - 1/k)    # normalizzato tra 0 e 1

# *** ENTROPIA ***
# Entropia di Shannon: 0=tutti uguali, log(k)=massima varietà
entropia = -np.sum(freq_rel * np.log(freq_rel))
entropia_norm = entropia / np.log(k)    # normalizzata tra 0 e 1
```

****Esempio reale (auto.csv):****

```
per_gini = auto.tipo_motore.value_counts(normalize=True)
k = len(per_gini)
gini_norm = (1 - np.sum(per_gini**2)) / (1 - 1/k)
```

6. Grafici (matplotlib.pyplot come plt)

```
import matplotlib.pyplot as plt

# Istogramma (distribuzione singola variabile)
dati["colonna"].plot.hist(bins=30)
plt.hist(dati["colonna"], bins=20, density=True)    # density=True normalizza

# Boxplot (confronto tra gruppi)
plt.boxplot([gruppo1, gruppo2], labels=["A", "B"])

# Scatter plot (relazione tra due variabili)
plt.scatter(dati["x"], dati["y"])
plt.xlabel("nome x"); plt.ylabel("nome y")

# Barre (categoriche)
plt.bar(x_values, y_values)

# Tortiera / Pie chart (proporzioni)
conteggi = dati["colonna"].value_counts()
plt.pie(conteggi, labels=conteggi.index, autopct='%1.1f%%')
```

```
# ECDF - funzione di ripartizione empirica (metodo 1)
rich = dati["colonna"].dropna().sort_values()
cdf = np.arange(1, len(rich)+1) / len(rich) * 100
plt.step(rich, cdf, where='post')
plt.xlabel("valori"); plt.ylabel("f (%)")

# ECDF - metodo 2 (con statsmodels)
from statsmodels.distributions.empirical_distribution import ECDF
ecdf = ECDF(dati["colonna"].dropna())
x = np.linspace(0, max(dati["colonna"].dropna()), 100)
plt.plot(x, ecdf(x))

# Confronto ECDF empirica vs CDF teorica
x = np.linspace(0, a, 100)
Fx = 3*x**2/(a**2) - 2*x**3/(a**3) # CDF teorica
plt.plot(x, ecdf(x)); plt.plot(x, Fx)
plt.show()

# Linee ausiliarie sul grafico
plt.axhline(y=95, color='red', linestyle='--')
plt.axvline(x=15, color='red', linestyle='--')
plt.grid(True)
plt.title("titolo")
plt.show() # mostra il grafico
plt.savefig("nome.png", dpi=100) # salva su file
```

7. Distribuzioni di probabilità (scipy.stats come st)

```
import scipy.stats as st

# *** NORMALE ***
st.norm.cdf(x, media, std) # P(X <= x) [funzione di ripartizione]
st.norm.pdf(x, media, std) # densità in x
st.norm.ppf(p, media, std) # valore con probabilità p (inverso CDF)
st.norm.cdf(z) # Phi(z) con standard (media=0, std=1)

# *** BINOMIALE ***
# Formula manuale: binom(n, k) * p^k * (1-p)^(n-k)
from scipy.special import binom
binom(n, k) # coefficiente binomiale

# Con scipy.stats:
st.binom.cdf(k, n, p) # P(X <= k)
st.binom.pmf(k, n, p) # P(X = k)

# *** POISSON ***
st.poisson.pmf(k, lambda_valore) # P(X = k)
st.poisson.cdf(k, lambda_valore) # P(X <= k)

# Anche a mano:
np.exp(-lambda_valore) * (lambda_valore**k) / factorial(k)
from scipy.special import factorial

# *** IPERGEOMETRICA ***
# P(X = k) = binom(N, k) * binom(M, n-k) / binom(N+M, n)
# dove: N=successi, M=fallimenti, n=estratti, k=successi estratti

# *** CHI-QUADRO ***
st.chi2.cdf(x, df=gradi_liberta)

# *** ESPONENZIALE ***
# P(T > t) = exp(-lambda * t)
# P(T <= t) = 1 - exp(-lambda * t)

# *** TEST STATISTICI ***
st.shapiro(dati) # Shapiro-Wilk (normalità)
st.normaltest(dati) # D'Agostino-Pearson (normalità)
st.ttest_1samp(dati, popmean=0) # T-test a un campione
st.f_oneway(a, b, c) # ANOVA a una via
```

```
st.pearsonr(x, y) # correlazione di Pearson + p-value
```

****Esempi reali:****

```
# P(X = 7) con Binomiale n=10, p=0.7
st.binom.pmf(7, 10, 0.7)

# P(X >= 2) con Binomiale = 1 - P(X <= 1)
1 - st.binom.cdf(1, 4, 0.6)

# P(X > 3) con Poisson(lambda=14)
1 - st.poisson.cdf(3, 14)

# P(Z < z) con Normale standard
st.norm.cdf((25 - 24.16) / 4.38)

# Phi_inversa: valore z con probabilità 0.9
st.norm.ppf(0.9)
```

8. Q-Q Plot (verifica distribuzione)

```
# Metodo 1: con statsmodels
import statsmodels.api as sm
sm.qqplot(dati["colonna"].dropna(), fit=True, line='45')
plt.show()
# fit=True: stima media e std dai dati per adattare la retta
# fit=False (default): usa N(0,1) come riferimento
# line='45': aggiunge la bisettrice (se i punti ci stanno distribuzione)

# Metodo 2: con scipy (non serve statsmodels)
st.probplot(dati["colonna"].dropna(), dist="norm", plot=plt)
plt.show()

# Q-Q plot contro Poisson (distribuzione diversa dalla normale)
st.probplot(dati["colonna"].dropna(), dist=st.poisson(lam), sparams=(), plot=plt)
plt.show()
```

9. Test di normalità

```
# Shapiro-Wilk: H0 = i dati sono normali
# Se p-value < 0.05 -> rifiuto H0 -> NON normale
stat, p_value = st.shapiro(dati["colonna"].dropna())
print(f"p-value = {p_value}")

# D'Agostino-Pearson: stessa logica
stat, p_value = st.normaltest(dati["colonna"].dropna())
```

10. Aggiungere colonne derivate

```
dati["nuova_colonna"] = 2 * dati["col1"] - dati["col2"]

# Creare variabili binarie da condizioni
dati["inverno"] = dati["mese"].isin([12, 1, 2]).astype(int)
```

11. ANOVA (confronto tra più gruppi)

```
# Separare i gruppi
a = dati[dati["arbitro"] == "A"]["differenza"]
b = dati[dati["arbitro"] == "B"]["differenza"]
c = dati[dati["arbitro"] == "C"]["differenza"]

# ANOVA a una via
f_stat, p_anova = st.f_oneway(a, b, c)
# p-value piccolo -> le medie dei gruppi sono significativamente diverse
```

12. Altre funzioni utili

```
# Array di valori equidistanti
np.arange(0, 10, 0.5)           # da 0 a 10 con passo 0.5
np.linspace(0, 10, 200)        # 200 punti equidistanti tra 0 e 10

# Stats su array numpy
np.mean(array); np.var(array); np.std(array)
np.unique(array, return_counts=True) # valori unici e loro frequenze
np.repeat(valori, frequenze)        # ricostruisce il campione

# Riprodurre valori da frequenze
campione = np.repeat(dati, freq)
np.median(campione)
```

13. TCL (Teorema Centrale del Limite)

```
# P(|stimatore - parametro| <= epsilon)
# = 2 * Phi(epsilon / (sigma/n)) - 1

# Esempio: P(|lambda - lambda| > 0.05) per Poisson
n = len(dati)
lam = dati.mean()
se = np.sqrt(lam / n)
z = 0.05 / se
prob = 2 * (1 - st.norm.cdf(z))      # probabilità che disti PIÙ di 0.05
```

14. Pattern comuni negli esami

Cosa chiede l'esercizio	Come si fa
Valori mancanti	<code>`dati.isnull().sum()`</code>
Frequenze relative	<code>`value_counts(normalize=True)`</code>
Tabella congiunta	<code>`pd.crosstab(..., normalize=True, margins=True)`</code>
Probabilità dalla tabella	Leggere cella corrispondente dalla crosstab
n per f% (ECDF)	<code>`dati["x"].quantile(f/100)`</code>
Relazione tra variabili	Scatter plot + <code>`.corr()`</code>
Distribuzione normale?	Istogramma + Q-Q plot + Shapiro test
Distribuzione Poisson?	Media Varianza + Q-Q plot vs Poisson
Stima lambda in Poisson	<code>`media campionaria`</code>
Boxplot comparativo	<code>`plt.boxplot([g1, g2], labels=[...])`</code>